

React(8)

React Router（ページ遷移）

Reactのページ遷移の概要

Reactでページ遷移を実現する主な方法は、**React Router**を使用することです。ReactはSPA（Single Page Application）なので、従来のWebサイトのように別のHTMLファイルに移動するのではなく、JavaScriptでコンテンツを切り替えます。

準備：React Routerのインストール

まず、プロジェクトにReact Routerをインストールします。

```
npm install react-router-dom
```

1 基本的なルーティング設定

まずは3つのページを作成して、ルーティングを設定してみましょう。

Step 1: ページコンポーネントを作成

以下の3つのシンプルなページを作成します。

```
// src/pages/Home.jsx
function Home() {
  return (
    <div>
      <h1>ホームページ</h1>
      <p>React Routerへようこそ！</p>
    </div>
  );
}

export default Home;
```

```
// src/pages/About.jsx
function About() {
```

```

    return (
      <div>
        <h1>アバウトページ</h1>
        <p>このアプリについての説明です。</p>
      </div>
    );
  }

  export default About;

```

```

// src/pages/Contact.jsx
function Contact() {
  return (
    <div>
      <h1>お問い合わせページ</h1>
      <p>お問い合わせフォームがここに入ります。</p>
    </div>
  );
}

export default Contact;

```

Step 2: App.jsxでルーティングを設定

4つの重要なコンポーネントを使ってルーティングを設定します。

```

// src/App.jsx
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
import Home from './pages/Home';
import About from './pages/About';
import Contact from './pages/Contact';

function App() {
  return (
    // 1 BrowserRouter : ルーティング機能を有効化する土台
    <BrowserRouter>
      { /* ナビゲーション部分 */ }
    </BrowserRouter>
  );
}

```

```

    <nav>
      {/* 4 Link : ページ遷移のリンクを作成 */}
      <Link to="/">ホーム</Link> |
      <Link to="/about">アバウト</Link> |
      <Link to="/contact">お問い合わせ</Link>
    </nav>

    {/* ページ表示部分 */}
    {/* 2 Routes : ルート定義をまとめるコンテナ */}
    <Routes>
      {/* 3 Route : URLとコンポーネントを紐付け */}
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
      <Route path="/contact" element={<Contact />} />
    </Routes>
  </BrowserRouter>
);
}

export default App;

```

重要な4つのコンポーネント：

- **BrowserRouter**：アプリ全体をラップし、ルーティング機能を有効化
- **Routes**：複数のRouteをまとめるコンテナ
- **Route**：URLパスとコンポーネントを紐付ける
- **Link**：ページ遷移用のリンク（aタグの代わり）

4つのコンポーネントの詳しい解説

それでは、App.jsxで使用している4つの重要なコンポーネントについて、1つずつ詳しく見ていきましょう。

1. BrowserRouter

役割：アプリ全体をルーティング機能で包み込むコンポーネント

```

<BrowserRouter>
  {/* ここに他のコンポーネントを配置 */}

```

```
</BrowserRouter>
```

- **必ず最も外側に配置**：ルーティング機能を使いたい全てのコンポーネントを `BrowserRouter` で囲む必要があります

「ルーティング機能を使えるようにするための準備」だと考えてください。この中に書かれたコンポーネントだけがページ遷移機能を使えるようになります。

- ブラウザのURLを変更し、戻る・進むボタンにも対応します
- **1つのアプリに1つだけ**：通常、App.jsxの最上位で1回だけ使用します



なぜBrowserRouterは1つのアプリに1つだけなのか？

`BrowserRouter` は、アプリ全体のURL状態、ブラウザ履歴、ナビゲーション情報を一元管理するコンポーネントです。複数配置すると、それぞれが独立してURLを管理しようとするため、矛盾や不具合が発生します。

Q: 複数のBrowserRouterを使ったらエラーになりますか？

A: 必ずしもエラーにはなりませんが、予期しない動作や不具合の原因になります。各BrowserRouterが独立してURLを管理しようとするため、ページ遷移が正しく動作しない可能性があります。

Q: 大規模なアプリでも1つだけですか？

A: はい、アプリケーションの規模に関わらず、BrowserRouterは1つだけです。ルート定義が増える場合は、`Routes`の中に多くの `Route` を追加したり、ネストしたルーティングを使用します。

2. Routes

役割：複数の `Route` をまとめて管理するコンテナ

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
  <Route path="/contact" element={<Contact />} />
</Routes>
```

- **複数のRouteをグループ化**： `Routes` の中に複数の `Route` を配置します
- **マッチするRouteを1つだけ表示**：現在のURLに一致する `Route` を探し、該当するコンポーネントだけを表示します

- **上から順番にチェック**：複数のパスがマッチする場合は、最初にマッチした `Route` が優先されます

3. Route

役割：URLのパスと表示するコンポーネントを紐付ける

```
<Route path="/about" element={<About />} />
```

2つの重要な属性

- `path`：URLのパスを指定します（例：`"/"`、`"/about"`、`"/user/:userId"`）
- `element`：そのパスにアクセスしたときに表示するコンポーネントを指定します

パスの書き方のパターン

```
// 通常のパス
<Route path="/about" element={<About />} />

// ルートパス（トップページ）
<Route path="/" element={<Home />} />

// 動的パラメータを含むパス（:をつける）
<Route path="/user/:userId" element={<UserProfile />} />

// ネストしたパス
<Route path="/products/detail" element={<ProductDetail />} />
```

「このURLにアクセスしたら、このページを表示してね」という指示書です。 `path` が「URL」、`element` が「表示する内容」と考えてください。

4. Link

役割：ページ遷移のためのリンクを作成する

```
<Link to="/about">アバウト</Link>
```

- **通常の `<a>` タグの代わり**：HTMLの `<a>` タグと似ていますが、ページ全体をリロードせずに遷移できます

- `to` 属性：遷移先のパスを指定します（`<a>` タグの `href` に相当）
- **SPAの利点を活かす**：クリックしてもページがリロードされないので、高速で滑らかな遷移が実現できます

通常のaタグとの違い：

```
// ❌ 通常のaタグ（使わない）
<a href="/about">アバウト</a> // ページ全体がリロードされる

// ✅ Linkコンポーネント（推奨）
<Link to="/about">アバウト</Link> // リロードせずに遷移
```

使い方のバリエーション：

```
// 基本的な使い方
<Link to="/">ホーム</Link>

// 動的なパスへのリンク
<Link to={` /user/${userId}`}>ユーザー詳細</Link>

// スタイルを適用
<Link to="/about" style={{ color: 'blue' }}>アバウト</Link>

// クラスを追加
<Link to="/contact" className="nav-link">お問い合わせ</Link>
```

「ページをリロードしないリンク」です。クリックすると、URLは変わりますが、ページ全体を読み込み直さずに表示内容だけが切り替わります。



なぜBrowserRouterは1つのアプリに1つだけなのか？

`BrowserRouter` はアプリ全体のURL状態と履歴を一元管理しています。複数配置すると、それぞれが独立してURLを管理しようとするため、ページ遷移が正しく動作しなくなります。アプリの規模が大きくなっても、`BrowserRouter` は常に1つだけ配置し、ルートを増やしたい場合は `Routes` の中に `Route` を追加していきます。

🎓 まとめ

コンポーネント	役割	使用箇所
BrowserRouter	ルーティング機能を有効化	App全体を囲む（最上位）
Routes	Route定義をまとめる	BrowserRouterの中
Route	URLとコンポーネントを紐付け	Routesの中
Link	ページ遷移のリンクを作成	BrowserRouterの中ならどこでも

これらの4つのコンポーネントを組み合わせることで、Reactアプリにページ遷移機能を実装できます！

2 useNavigate：プログラマ的なページ遷移

Link コンポーネントは、ナビゲーションメニューのような常に画面に表示されている固定リンクに適しています。しかし、「ボタンをクリックしたとき」「フォーム送信後に別のページへ移動したいとき」「条件によって遷移先を変えたいとき」など、JavaScriptの処理の中でページ遷移を行いたい場面では useNavigate を使います。

Link と useNavigate の使い分け

使い分け	使用するもの	具体例
固定のリンクを表示したい	Link	ナビゲーションメニュー、サイドバー
イベントや条件に応じて遷移したい	useNavigate	ボタンクリック、ログイン後の遷移、エラー時のリダイレクト

基本的な使い方

Home.jsxにボタンとイベント処理を追加してみましょう。

```
// src/pages/Home.jsx
import { useNavigate } from 'react-router-dom';

function Home() {
  const navigate = useNavigate();

  const goToAbout = () => {
    navigate('/about');
  };
}
```

```

return (
  <div>
    <h1>ホームページ</h1>
    <p>React Routerへようこそ！</p>
    <button onClick={goToAbout}>アバウトページへ</button>
  </div>
);
}

export default Home;

```

useNavigateの便利な使い方

```

const navigate = useNavigate();

// 指定したパスへ遷移
navigate('/about');

// 前のページに戻る（ブラウザの「戻る」ボタンと同じ）
navigate(-1);

// 次のページへ進む（ブラウザの「進む」ボタンと同じ）
navigate(1);

```

`navigate(-1)` は特によく使います。詳細ページから一覧ページに「戻る」ボタンを実装する場面で活躍します。

3 useParams：URLパラメータの取得

Webアプリでは、URLの一部をデータとして使いたい場面がよくあります。例えば `/user/123` というURLにアクセスしたとき、「123」という部分を取り出してそのユーザーの情報を表示したい、というケースです。

React Routerでは、`useParams` というフックを使って、URLに埋め込まれた値（パラメータ）を簡単に取得できます。

Step 1: 動的ルートを追加

まず、URLの一部を**変数**として受け取れるルートを定義します。`:userId` の部分が「ここには何でも入る」という意味のプレースホルダーです。


```
// src/App.jsx に追加
import UserProfile from './pages/UserProfile';

// Routes内に追加
<Route path="/user/:userId" element={<UserProfile />} />
```

`:userId` のコロン (`:`) がポイントです。これがあることで、React Routerは「この部分は固定の文字列ではなく、動的に変わる値だ」と認識します。

Step 2: useParamsでパラメータを取得する

次に、遷移先のコンポーネントで `useParams` を呼び出すと、URLに含まれるパラメータをオブジェクトとして受け取れます。

```
// src/pages/UserProfile.jsx
import { useParams } from 'react-router-dom';

function UserProfile() {
  const { userId } = useParams();

  return (
    <div>
      <h1>ユーザープロフィール</h1>
      <p>ユーザーID: {userId}</p>
    </div>
  );
}

export default UserProfile;
```

動作の確認

アクセスするURL	userId の値
/user/123	"123"
/user/yamada	"yamada"
/user/42	"42"

URLを変えるだけで、同じコンポーネント (`UserProfile`) が異なるデータを表示できる仕組みです。これにより、ユーザーごとに別々のページを用意する必要がなく

なります。

注意： `useParams` で取得した値は常に**文字列**です。数値として使いたい場合は `Number(userId)` のように変換が必要です。 `Number()` はJavaScriptに最初から用意されている**組み込み関数**です。文字列を数値に変換するときに使います。



似た方法として `parseInt()` もありますが、違いがあります。

`Number("123")` は 123 を返し、`parseInt("123")` も 123 を返します。ただし `Number("123abc")` は NaN（変換失敗）になるのに対して、`parseInt("123abc")` は先頭の数字部分だけ読み取って 123 を返します。

実践課題：商品紹介アプリを作ろう

新規プロジェクト `react08-kadai` を作成してください。

課題の概要

この課題では、今回学んだReact Routerの機能を組み合わせて、複数ページを持つ商品紹介アプリを作成します。以下の5つのステップに沿って、順番に実装してみましょう。

以下のファイルには、あらかじめ土台（テンプレート）が用意されています。各ファイルの `TODO` コメントを手がかりに、自分でコードを書いて完成させてください。

ファイル	役割
<code>src/App.jsx</code>	ルーティング設定とナビゲーションメニュー
<code>src/pages/Home.jsx</code>	ホームページ
<code>src/pages/Products.jsx</code>	商品一覧ページ
<code>src/pages/ProductDetail.jsx</code>	商品詳細ページ
<code>src/pages/Contact.jsx</code>	お問い合わせページ

Step 1：ルーティングを設定する（ `App.jsx` ）

`<Routes>` の中に4つの `<Route>` を定義してください。

- `/` → Home コンポーネント
- `/products` → Products コンポーネント

- /product/:productId → ProductDetail コンポーネント
- /contact → Contact コンポーネント

💡 :productId の部分は動的パラメータです。第3章で学んだ useParams で取得できます。

Step 2：ナビゲーションメニューを作る（App.jsx）

<nav> の中に <Link> を使って、ホーム・商品一覧・お問い合わせへのリンクを3つ作成してください。

💡 <a> タグではなく <Link> を使うのがポイントです。理由を思い出せない人は、第1章を復習しましょう。

Step 1・2 のテンプレート（src/App.jsx）

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
import Home from './pages/Home';
import Products from './pages/Products';
import ProductDetail from './pages/ProductDetail';
import Contact from './pages/Contact';

function App() {
  return (
    <BrowserRouter>
      {/* TODO: ナビゲーションメニューを作成 */}
      {/* ヒント: <nav>の中に<Link>を使って3つのページへのリンクを作りましょう */}
      <nav>

      </nav>

      <Routes>
        {/* TODO: 4つのルートを定義 */}
        {/* ヒント: ホーム(/), 商品一覧(/products), 商品詳細(/product/:productId), お問い合わせ(/contact) */}

        </Routes>
      </BrowserRouter>
    );
}
```

```
    </BrowserRouter>
  );
}
```

```
export default App;
```

src/pages/Home.jsx

```
function Home() {
  // TODO: ホームページの内容を自由に作成してみましょう
  return (
    <div>
      <h1>ホームページ</h1>
    </div>
  );
}

export default Home;
```

src/pages/Contact.jsx

```
function Contact() {
  // TODO: お問い合わせページの内容を自由に作成してみましょう
  return (
    <div>
      <h1>お問い合わせ</h1>
    </div>
  );
}

export default Contact;
```

Step 3：商品一覧から詳細ページへ遷移する（**Products.jsx**）

useNavigate を使って、各商品の「商品詳細を見る」ボタンをクリックしたときに /product/1 のようなURLへ遷移する処理を書いてください。

💡 サンプル商品データはすでに用意されています。map で繰り返し表示し、ボタンの onClick で navigate() を呼びましょう。

テンプレート (src/pages/Products.jsx)

```
// TODO0: useNavigateをインポートしましょう

function Products() {
  // TODO0: useNavigateを使って、navigate関数を取得しましょう

  // サンプル商品データ
  const products = [
    { id: 1, name: 'ノートパソコン' },
    { id: 2, name: 'ワイヤレスマウス' },
    { id: 3, name: 'USBキーボード' },
  ];

  return (
    <div>
      <h1>商品一覧</h1>
      { /* TODO0: products配列をmapで回して、各商品の名前と「商品詳細を見る」ボタンを表示 */ }
      { /* ヒント: ボタンのonClickでnavigate(`/product/${product.id}`)を呼びましょう */ }

    </div>
  );
}

export default Products;
```

Step 4・5：商品詳細ページを完成させる (**ProductDetail.jsx**)

このステップでは、1つのファイルで2つの機能を実装します。

Step 4： useParams を使って、URLに含まれる productId を取得し、画面に表示してください。

💡 /product/2 にアクセスしたとき、「商品ID: 2」のように表示されればOKです。

Step 5 : useNavigate を使って、「戻る」ボタンを押すと前のページに戻れるようにしてください。

💡 navigate(-1) で、ブラウザの「戻る」ボタンと同じ動作になります。

テンプレート (src/pages/ProductDetail.jsx)

```
// TODO: useParamsとuseNavigateをインポートしましょう

function ProductDetail() {
  // TODO: useParamsでproductIdを取得しましょう
  // TODO: useNavigateでnavigate関数を取得しましょう

  return (
    <div>
      <h1>商品詳細ページ</h1>
      {/* TODO: 取得したproductIdを表示しましょう */}

      {/* TODO: 「戻る」ボタンを作成し、navigate(-1)で前のページに
      戻れるようにしましょう */}

    </div>
  );
}

export default ProductDetail;
```

実践課題：解答例

src/App.jsx

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
import Home from './pages/Home';
import Products from './pages/Products';
import ProductDetail from './pages/ProductDetail';
import Contact from './pages/Contact';
```

```

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">ホーム</Link> |
        <Link to="/products">商品一覧</Link> |
        <Link to="/contact">お問い合わせ</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/products" element={<Products />} />
        <Route path="/product/:productId" element={<Product
Detail />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;

```

src/pages/Home.jsx

```

function Home() {
  return (
    <div>
      <h1>ホームページ</h1>
      <p>商品紹介アプリへようこそ！</p>
    </div>
  );
}

export default Home;

```

src/pages/Products.jsx

```

import { useNavigate } from 'react-router-dom';

function Products() {
  const navigate = useNavigate();

  const products = [
    { id: 1, name: 'ノートパソコン' },
    { id: 2, name: 'ワイヤレスマウス' },
    { id: 3, name: 'USBキーボード' },
  ];

  return (
    <div>
      <h1>商品一覧</h1>
      {products.map((product) => (
        <div key={product.id}>
          <p>{product.name}</p>
          <button onClick={() => navigate(`/product/${product.id}`}>
            商品詳細を見る
          </button>
        </div>
      ))}
    </div>
  );
}

export default Products;

```

src/pages/ProductDetail.jsx

```

import { useParams, useNavigate } from 'react-router-dom';

function ProductDetail() {
  const { productId } = useParams();
  const navigate = useNavigate();

  return (

```



```

    <div>
      <h1>商品詳細ページ</h1>
      <p>商品ID: {productId}</p>
      <button onClick={() => navigate(-1)}>戻る</button>
    </div>
  );
}

export default ProductDetail;

```

src/pages/Contact.jsx

```

function Contact() {
  return (
    <div>
      <h1>お問い合わせ</h1>
      <p>お問い合わせはメールでお願いします。</p>
    </div>
  );
}

export default Contact;

```